



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

CSA0403 -OPERATING SYSTEM

GUIDE BY: CHARLYN PUSHPA LATHA G

PRESENTED BY:

NAME:SIVA SUBRAMANIYAM B

REGISTER NO:192572089

main.c

Run

Clear

```

1 #include <stdio.h>
2 int main()
3 {
4     int A[100][4];
5     int i, j, n, total = 0, index, temp; float
6     avg_wt, avg_tat;
7     printf("Enter number of process: ");
8     scanf("%d", &n);
9     printf("Enter Burst Time:\n");
10    for (i= 0; i < n; i++) {
11        printf("P%d: ", i + 1);
12        scanf("%d", &A[i][1]);
13        A[i][0] = i + 1;
14    }
15    for (i= 0; i < n; i++) {

```

Enter number of process: 3
Enter Burst Time:
P1: 10
P2: 7
P3: 8
P BT WT TAT
P2 7 0 7
P3 8 7 15
P1 10 15 25
Average Waiting Time= 7.333333
AverageTurnaround Time= 15.666667

=== Code Execution Successful ===

main.c

Run

Clear

```

37     avg_wt = (float)total/n;
38     total=0;
39     printf("nProcesst Burst Time tWaiting
40         TimeTurnaround Time\n");
41     for(i=0;i<n;i++)
42     {
43         tat[i]=bt[i]+wt[i];
44         total+=tat[i];
45         printf("np%dtt %dtt %dttt%d",p[i],bt[i],wt[i]
46             ,tat[i]);
47     }
48     avg_tat=(float)total/n;
49     printf("nnAverage Waiting Time=%f",avg_wt);
50     printf("nAverage Turnaround Time=%fn",avg_tat);

```

Enter number of process:4
nEnter Burst Time:
p1:5
p2:7
p3:8
p4:9
nProcesst Burst Time tWaiting TimetTurnaround Time
np1tt 5tt 0ttt5np2tt 7tt 5ttt12np3tt 8tt 12ttt20np4tt
9tt 20ttt29nnAverage Waiting Time=9.250000nAverage
Turnaround Time=16.500000n

=== Code Execution Successful ===

Programiz
C Online Compiler

main.c

```

32  scanf("%d", &process[i].priority);
33  ASCII_number++;
34  }
35  // Sort processes by priority (Highest Priority First)
36  for (int i = 0; i < number_of_process; i++) {
37      position = i;
38      for (int j = i + 1; j < number_of_process; j++) {
39          if (process[j].priority > process[position].priority) {
40              position = j;
41          }
42      }
43      temp_process = process[i];
44      process[i] = process[position];
45      process[position] = temp_process;
46  }
47  // Calculate waiting time
48  process[0].waiting_time = 0;
49  for (int i = 1; i < number_of_process; i++) {
50      process[i].waiting_time = 0;
51      for (int j = 0; j < i; j++) {
52          process[i].waiting_time += process[j].burst_time;
53      }
54      total += process[i].waiting_time;
55  }
56  average_waiting_time = (float)total / (float)number_of_process;
57  total = 0;
58  printf("\n\nProcess_name \t Burst Time \t Waiting Time \t Turnaround Time\n");
59  printf("\n\n");
60  // Calculate turnaround time and display details
61  for (int i = 0; i < number_of_process; i++) {
62      process[i].turn_around_time = process[i].burst_time + process[i].waiting_time;
63      total += process[i].turn_around_time; // Fixed: Added missing logic to sum turnaround times
64      printf("\n\nProcess No: %d \t Burst Time: %d \t Waiting Time: %d \t Turnaround Time: %d\n",
65          process[i].process_name,
66          process[i].burst_time,
67          process[i].waiting_time,
68          process[i].turn_around_time);
69  }
70  printf("\n\n");
71  average_turnaround_time = (float)total / (float)number_of_process;
72  printf("\n\nAverage Waiting Time: %f", average_waiting_time);
73  printf("\n\nAverage Turnaround Time: %f\n", average_turnaround_time);
74  return 0;
75  }
76  }

```

Output

Enter the total number of Processes: 3

Please Enter the Burst Time and Priority of each process:

Enter the details of the process A
Enter the burst time: 3
Enter the priority: 2

Enter the details of the process B
Enter the burst time: 5
Enter the priority: 1

Enter the details of the process C
Enter the burst time: 7
Enter the priority: 3

Process_name	Burst Time	Waiting Time	Turnaround Time
C	7	0	7
A	3	7	10
B	5	10	15

Average Waiting Time: 5.666667
Average Turnaround Time: 10.666667

=== Code Execution Successful ===

main.c

```

52  // Cycle through processes, checking arrival time
53  if(i == NOP - 1)
54  {
55      i = 0;
56  }
57  else if(at[i+1] <= sum)
58  {
59      i++;
60  }
61  else
62  {
63      i = 0;
64  }
65  }
66  // Corrected variable usage and precision output
67  avg_wt = wt * 1.0 / NOP;
68  avg_tat = tat * 1.0 / NOP;
69  printf("\n\nAverage Waiting Time: %.2f", avg_wt);
70  printf("\n\nAverage Turn Around Time: %.2f\n", avg_tat);
71  return 0;
72  }
73  }
74  }
75  }
76  }

```

close ad [x]

input

Arrival time is: 5

Burst time is: 4

Enter the Time Quantum for the process: 4

Process No	Burst Time	TAT	Waiting Time
Process No[1]	2	-2	-4

^[[21~

Online C Compiler × OS ppt5 - Classroom × CSA04 OS LAB MA × #include <stdio.h> × Online C Compiler × + Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/ipc.h>
6 #include <sys/shm.h>
7
8 #define SHM_SIZE 1024
9
10 int main() {
11     // Create the file if it does not exist
12     FILE *fp = fopen("shmfile", "w");
13     if (fp == NULL) fclose(fp);
14
15     key_t key = ftok("shmfile", 65);
16     if (key == -1) {
17         perror("ftok");
18         exit(EXIT_FAILURE);
19     }
20
21     int shmid = shmget(key, SHM_SIZE, IPC_CREAT | 0666);
22     if (shmid == -1) {
23         perror("shmget");
24         exit(EXIT_FAILURE);
25     }
26
27     char *shm_ptr = (char*)shmat(shmid, NULL, 0);
28     if (shm_ptr == (char*)-1) {
29         perror("shmat");
30         exit(EXIT_FAILURE);
31     }
32
33     strcpy(shm_ptr, "Hello, shared memory!");
34
35     // Print BEFORE detaching
36     printf("Data written to shared memory: %s\n", shm_ptr);
37
38     if (shmat(shm_ptr, 0, 0) == -1) {
39         perror("shmat");
40         exit(EXIT_FAILURE);
41     }
42
43     if (shmat(shmid, IPC_NOWAIT, 0) == -1) {
44         perror("shmat");
45         exit(EXIT_FAILURE);
46     }
47
48     return 0;
49 }
```

Output

frak: No such file or directory
--- Code Exited With Errors ---

Online C Compiler × OS ppt5 - Classroom × CSA04 OS LAB MA × #include <stdio.h> × Online C Compiler × + Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6 #include <sys/ipc.h>
7 #include <sys/msg.h>
8
9 struct message {
10     long msg_type;
11     char msg_text[100];
12 };
13
14 int main() {
15     // Generate a unique key for the message queue
16     key_t key = ftok("msgfile", 65);
17
18     // Create a new message queue (or get the identifier of an existing one)
19     int msgid = msgget(key, IPC_CREAT | 0666);
20     if (msgid == -1) {
21         perror("msgget");
22         exit(EXIT_FAILURE);
23     }
24
25     struct message msg;
26     msg.msg_type = 1; // Message type (can be any positive number)
27
28     // Producer: Send a message to the message queue
29     strcpy(msg.msg_text, "Hello, message queue!");
30     if (msgsnd(msgid, (void*)&msg, sizeof(msg.msg_text), IPC_NOWAIT) == -1) {
31         perror("msgsnd");
32         exit(EXIT_FAILURE);
33     }
34     printf("Producer: Data sent to message queue: %s\n", msg.msg_text);
35
36     // Consumer: Receive a message from the message queue
37     if (msgrcv(msgid, (void*)&msg, sizeof(msg.msg_text), 1, 0) == -1) {
38         perror("msgrcv");
39         exit(EXIT_FAILURE);
40     }
41     printf("Consumer: Data received from message queue: %s\n", msg.msg_text);
42
43     // Remove the message queue
44     if (msgctl(msgid, IPC_RMID, 0) == -1) {
45         perror("msgctl");
46         exit(EXIT_FAILURE);
47     }
48
49     return 0;
50 }
```

Output

Producer: Data sent to message queue: Hello, message queue!
Consumer: Data received from message queue: Hello, message queue!
--- Code Execution Successful ---

Online C Compiler - OS ppt5 - Classroom - CSA04 OS LAB MA - #include <stdio.h> - Online C Compiler - Ask Gemini - programiz.com/c-programming/online-compiler/

```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3
4 void* threadFunction(void* arg) {
5     char* message = (char*)arg;
6     printf("Hello\n", message);
7     return NULL;
8 }
9
10 int main() {
11     pthread_t thread1, thread2;
12     char* message1 = "Hello from Thread 1!";
13     char* message2 = "Hello from Thread 2!";
14
15     // Create threads
16     pthread_create(&thread1, NULL, threadFunction, (void*)message1);
17     pthread_create(&thread2, NULL, threadFunction, (void*)message2);
18
19     // Wait for threads to complete
20     pthread_join(thread1, NULL);
21     pthread_join(thread2, NULL);
22
23     return 0;
24 }
25
```

Output

```
Hello from Thread 1!
Hello from Thread 2!

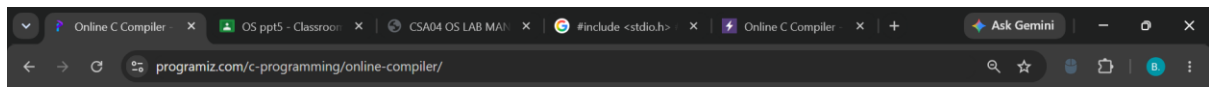
=== Code Execution Successful ===
```

Online C Compiler - OS ppt5 - Classroom - CSA04 OS LAB MA - #include <stdio.h> - Online C Compiler - Ask Gemini - programiz.com/c-programming/online-compiler/

```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #define NUM_PHILOSOPHERS 5
5
6 pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
7
8 void* philosopherLifeCycle(void* arg) {
9     int id = *((int*)arg);
10    int left_chopstick = id;
11    int right_chopstick = (id + 1) % NUM_PHILOSOPHERS;
12
13    while (1) {
14        // Think
15        printf("Philosopher %d is thinking...\n", id);
16
17        // Pick up chopsticks
18        pthread_mutex_lock(&chopsticks[left_chopstick]);
19        pthread_mutex_lock(&chopsticks[right_chopstick]);
20
21        // Eat
22        printf("Philosopher %d is eating...\n", id);
23        sleep(rand() % 3 + 1); // Eating time
24
25        // Put down chopsticks
26        pthread_mutex_unlock(&chopsticks[left_chopstick]);
27        pthread_mutex_unlock(&chopsticks[right_chopstick]);
28
29        // Repeat the cycle
30    }
31 }
32
33 int main() {
34     pthread_t philosophers[NUM_PHILOSOPHERS];
35     int philosopher_id[NUM_PHILOSOPHERS];
36
37     // Initialize mutex locks
38     for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
39         pthread_mutex_init(&chopsticks[i], NULL);
40     }
41
42     // Create philosopher threads
43     for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
44         philosopher_id[i] = i;
45         pthread_create(&philosophers[i], NULL, philosopherLifeCycle, (void*)&philosopher_id[i]);
46     }
47
48     // Wait for threads to finish (although they run indefinitely)
49     for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
50         pthread_join(philosophers[i], NULL);
51     }
52 }
53
```

Output

```
Philosopher 0 is thinking...
Philosopher 0 is eating...
Philosopher 1 is thinking...
Philosopher 2 is thinking...
Philosopher 2 is eating...
Philosopher 3 is thinking...
Philosopher 3 is eating...
Philosopher 4 is thinking...
Philosopher 4 is eating...
Philosopher 0 is thinking...
Philosopher 0 is eating...
Philosopher 1 is thinking...
Philosopher 1 is eating...
Philosopher 2 is thinking...
Philosopher 2 is eating...
Philosopher 3 is thinking...
Philosopher 3 is eating...
Philosopher 4 is thinking...
Philosopher 4 is eating...
```

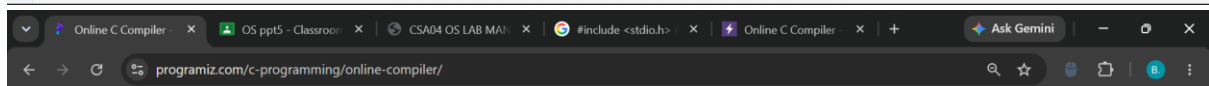


Programiz C Online Compiler

```
main.c
1 #include <stdio.h>
2
3 void bestfit(int mp[], int p[], int m, int n) {
4     int j = 0;
5     for (int i = 0; i < m; i++) {
6         if (mp[i] > p[j]) {
7             printf("No fits in %d", p[j], mp[i]);
8             mp[i] = mp[i] - p[j];
9             i = i - 1;
10        }
11    }
12    for (int i = 0; i < m; i++) {
13        printf("Process %d must wait for its process", p[i]);
14    }
15 }
16
17 void rsort(int a[], int n) {
18     for (int i = 0; i < n; i++) {
19         for (int j = 0; j < n; j++) {
20             if (a[i] > a[j]) {
21                 int t = a[i];
22                 a[i] = a[j];
23                 a[j] = t;
24             }
25         }
26     }
27 }
28
29 void sort(int a[], int n) {
30     for (int i = 0; i < n; i++) {
31         for (int j = 0; j < n; j++) {
32             if (a[i] > a[j]) {
33                 int t = a[i];
34                 a[i] = a[j];
35                 a[j] = t;
36             }
37         }
38     }
39 }
40
41 void firstfit(int mp[], int p[], int m, int n) {
42     sort(mp, m);
43     sort(p, n);
44     bestfit(mp, p, m, n);
45 }
46
47 void worstfit(int mp[], int p[], int m, int n) {
48     rsort(mp, m);
49     sort(p, n);
50     bestfit(mp, p, m, n);
51 }
52
53 int main() {
54     int m, n;
55     printf("Enter the number of memory partitions: ");
56     scanf("%d", &m);
57     int mp[m];
58     printf("Enter the sizes of memory partitions: ");
59     for (int i = 0; i < m; i++) {
60         scanf("%d", &mp[i]);
61     }
62     printf("Enter the number of processes: ");
63     scanf("%d", &n);
64     int p[n];
65     printf("Enter the sizes of processes: ");
66     for (int i = 0; i < n; i++) {
67         scanf("%d", &p[i]);
68     }
69     printf("1. Firstfit 2. Bestfit 3. worstfit\n");
70     printf("Enter your choice: ");
71     int choice;
72     scanf("%d", &choice);
73     switch (choice) {
74         case 1: firstfit(mp, p, m, n); break;
75         case 2: bestfit(mp, p, m, n); break;
76         case 3: worstfit(mp, p, m, n); break;
77     }
78     printf("Code Execution Successful ==>");
79 }
```

Output

```
Number of memory partition : 3
Number of process : 4
Enter the memory partitions :
300
300
400
Enter process size :
50
100
150
70
1. Firstfit 2. Bestfit 3. worstfit
Enter your choice : 1
50 fits in 300
100 fits in 300
150 fits in 400
70 fits in 300
Code Execution Successful ==>
```



Programiz C Online Compiler

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 #define BUFFER_SIZE 4096
6
7 void copy() {
8     const char *sourcefile = "C:/Users/itsak/OneDrive/Desktop/sasi.txt";
9     const char *destination_file = "C:/Users/itsak/OneDrive/Desktop/sk.txt";
10    int source_fd = open(sourcefile, O_RDONLY);
11    int dest_fd = open(destination_file, O_WRONLY | O_CREAT | O_TRUNC, 0666);
12    char buffer[BUFFER_SIZE];
13    ssize_t bytesread, byteswritten;
14    while ((bytesread = read(source_fd, buffer, BUFFER_SIZE)) > 0) {
15        byteswritten = write(dest_fd, buffer, bytesread);
16    }
17    close(source_fd);
18    close(dest_fd);
19    printf("File copied successfully.\n");
20 }
21
22 void create() {
23     char path[100];
24     FILE *fp;
25     fp = fopen("C:/Users/itsak/OneDrive/Desktop/sasi.txt", "w");
26     if (fp != NULL) {
27         fclose(fp);
28     }
29     printf("File created successfully.\n");
30 }
31
32 int main() {
33     int n;
34     printf("1. Create 2. Copy 3. Delete\n");
35     printf("Enter your choice: ");
36     scanf("%d", &n);
37     switch (n) {
38         case 1: create(); break;
39         case 2: copy(); break;
40         case 3: remove("C:/Users/itsak/OneDrive/Desktop/sasi.txt"); break;
41     }
42 }
```

Output

```
1. Create 2. Copy 3. Delete
Enter your choice: 2
File copied successfully.
Code Execution Successful ==>
```


Online C Compiler - OS ppt5 - Classroom - CSA04 OS LAB MA - #include <stdio.h> - Online C Compiler - Ask Gemini - programiz.com/c-programming/online-compiler/ - programiz PRO

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main() {
6     char mainDirPath[] = "C:/Users/itsk/OneDrive/Desktop";
7     char subDirectory[] = "os";
8     char fileName[] = "example.txt";
9     char filePath[200];
10    char mainDirPath[200];
11
12    // Build the directory path and target file path strings
13    sprintf(mainDirPath, "%s\\%s", mainDirPath, subDirectory);
14    sprintf(filePath, "%s\\%s", mainDirPath, fileName);
15
16    // Open and write to the target file
17    FILE *file = fopen(filePath, "w");
18    if (file == NULL) {
19        printf("Error creating file.\n");
20        return 1;
21    }
22    fprintf(file, "This is an example file content.");
23    fclose(file); // Critical Resource Fix
24    printf("File created successfully: %s\n", filePath); // Log Format Fix
25    return 0;
26 }
```

Output
Error creating file.
--- Code Exited With Errors ---

Online C Compiler - OS ppt5 - Classroom - CSA04 OS LAB MA - #include <stdio.h> - Online C Compiler - Ask Gemini - programiz.com/c-programming/online-compiler/ - programiz PRO

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Employee {
5     int empId;
6     char empName[50];
7     float empSalary;
8 };
9
10 int main() {
11     FILE *filePtr;
12     struct Employee emp;
13
14     // Open file for reading/writing in binary format
15     filePtr = fopen("employee.dat", "rb+");
16     if (filePtr == NULL) {
17         filePtr = fopen("employee.dat", "wb+");
18         if (filePtr == NULL) {
19             printf("Error creating the file.\n");
20             return 1;
21         }
22     }
23
24     int choice;
25
26     do {
27         printf("\nEmployee Database Menu\n");
28         printf("1. Add Employee\n");
29         printf("2. Display Employee Details\n");
30         printf("3. Update Employee Details\n");
31         printf("4. Exit\n");
32         printf("Enter your choice: ");
33         scanf("%d", &choice);
34
35         switch (choice) {
36             case 1:
37                 printf("Enter Employee ID: ");
38                 scanf("%d", &emp.empId);
39                 printf("Enter Employee Name: ");
40                 scanf("%s", emp.empName);
41                 printf("Enter Employee Salary: ");
42                 scanf("%f", &emp.empSalary);
43
44                 fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee), SEEK_SET);
45                 fwrite(&emp, sizeof(struct Employee), 1, filePtr);
46                 printf("Employee details added successfully.\n");
47                 break;
48
49             case 2:
50                 printf("Enter Employee ID to display: ");
51                 scanf("%d", &emp.empId);
52
53                 fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee), SEEK_SET);
```

Output
Error creating the file.
--- Code Exited With Errors ---

Online C Compiler - x OS ppt5 - Classroom - x CSA04 OS LAB MAH - x #include <stdio.h> - x Online C Compiler - x + Ask Gemini - - D O X

onlinegdb.com/online_c_compiler

OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login

GET SKILLS IN
JAVA PROGRAMMING
CODECARTOON

About • FAQ • Docs • Blog • Terms of Use •
Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

main.c

```
1 #include <stdio.h>
2
3 #define MAX_PROCESSES 5
4 #define MAX_RESOURCES 3
5
6 int is_safe();
7
8 int available[MAX_RESOURCES] = {3, 3, 2}; // Available instances of each res
9
10 int maximum[MAX_PROCESSES][MAX_RESOURCES] = {
11     {7, 5, 3},
12     {3, 2, 2},
13     {9, 0, 2},
14     {2, 2, 2},
15     {4, 3, 3}
16 };
17
18 int allocation[MAX_PROCESSES][MAX_RESOURCES] = {
19     {0, 1, 0},
20     {2, 0, 0},
21     {3, 0, 2},
22     {2, 1, 1},
23     {0, 0, 2}
24 };
25
```

Download Current File

close ad [x]

input

Enter resource request (e.g., 0 1 0): 1
0
1
Request denied. System is not in safe state.
...Program finished with exit code 0
Press ENTER to exit console.

Online C Compiler - x OS ppt5 - Classroom - x CSA04 OS LAB MAH - x #include <stdio.h> - x Online C Compiler - x + Ask Gemini - - D O X

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <semaphore.h>
4 #include <windows.h>
5
6 #define BUFFER_SIZE 5
7 #define MAX_ITEMS 10 // Maximum number of items to be produced/consumed
8
9 int buffer[BUFFER_SIZE];
10 sem_t empty, full;
11 int produced_items = 0, consumed_items = 0;
12
13 void* producer(void* arg) {
14     while (produced_items < MAX_ITEMS) {
15         sem_wait(&empty);
16
17         // Critical section: add item to buffer
18         for (int i = 0; i < BUFFER_SIZE; ++i) {
19             if (buffer[i] == 0) {
20                 buffer[i] = produced_items++;
21                 printf("Produced %d\n", buffer[i]);
22                 produced_items++;
23                 break;
24             }
25         }
26
27         sem_post(&full);
28         Sleep(1); // Sleep for a while
29     }
30     return NULL;
31 }
32
33 void* consumer(void* arg) {
34     while (consumed_items < MAX_ITEMS) {
35         sem_wait(&full);
36
37         // Critical section: remove item from buffer
38         for (int i = 0; i < BUFFER_SIZE; ++i) {
39             if (buffer[i] != 0) {
40                 printf("Consumed %d\n", buffer[i]);
41                 buffer[i] = 0;
42                 consumed_items++;
43                 break;
44             }
45         }
46
47         sem_post(&empty);
48         Sleep(2); // Sleep for a while
49     }
50     return NULL;
51 }
52
53 int main() {
```

Output

Error: /tmp/RJt9d9Pb/main.c:4:10: fatal error: Windows.h: No such file or directory
4 | #include <Windows.h>
|
| compilation terminated.
--- Code Exited With Errors ---

Online C Compiler - OS ppt5 - Classroom - CSA04 OS LAB MA - #include <stdio.h> - Online C Compiler - Ask Gemini - programiz.com/c-programming/online-compiler/

```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3
4 // Shared variables
5 int counter = 0;
6 pthread_mutex_t mutex;
7
8 // Function to be executed by threads
9 void *threadFunction(void *arg) {
10     int i;
11     for (i = 0; i < 1000000; ++i) {
12         pthread_mutex_lock(&mutex);
13         counter++;
14         pthread_mutex_unlock(&mutex);
15     }
16     return NULL;
17 }
18
19 int main() {
20     pthread_mutex_init(&mutex, NULL);
21     pthread_t thread1, thread2;
22     pthread_create(&thread1, NULL, threadFunction, NULL);
23     pthread_create(&thread2, NULL, threadFunction, NULL);
24
25     // Wait for the threads to finish
26     pthread_join(thread1, NULL);
27     pthread_join(thread2, NULL);
28
29     // Destroy the mutex
30     pthread_mutex_destroy(&mutex);
31
32     // Print the final value of the counter
33     printf("Final counter value: %d\n", counter);
34
35     return 0;
36 }
```

Output

Final counter value: 2000000

Code Execution Successful

Online C Compiler - OS ppt5 - Classroom - CSA04 OS LAB MA - #include <stdio.h> - Online C Compiler - Ask Gemini - programiz.com/c-programming/online-compiler/

```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4
5 sem_t mutex, writeLock;
6 int data = 0, readersCount = 0;
7
8 void *reader(void *arg) {
9     int i = 0;
10    while (i < 10) {
11        sem_wait(&mutex);
12        readersCount++;
13        if (readersCount == 1) {
14            sem_wait(&writeLock);
15        }
16        sem_post(&mutex);
17
18        // Reading operation
19        printf("Reader reads data: %d\n", data);
20
21        sem_wait(&mutex);
22        readersCount--;
23        if (readersCount == 0) {
24            sem_post(&writeLock);
25        }
26        sem_post(&mutex);
27
28        i++;
29    }
30    return NULL;
31 }
32
33 void *writer(void *arg) {
34     int i = 0;
35     while (i < 10) {
36         sem_wait(&writeLock);
37
38         // Writing operation
39         data++;
40         printf("Writer writes data: %d\n", data);
41
42         sem_post(&writeLock);
43
44         i++;
45     }
46     return NULL;
47 }
48
49 int main() {
50     pthread_t readers, writers;
51     sem_init(&mutex, 0, 1);
52     sem_init(&writeLock, 0, 1);
53 }
```

Output

Reader reads data: 0
Reader reads data: 0
Reader reads data: 0
Reader reads data: 0
Reader reads data: 0
Reader reads data: 0
Reader reads data: 0
Reader reads data: 0
Reader reads data: 0
Reader reads data: 0
Writer writes data: 1
Writer writes data: 2
Writer writes data: 3
Writer writes data: 4
Writer writes data: 5
Writer writes data: 6
Writer writes data: 7
Writer writes data: 8
Writer writes data: 9
Writer writes data: 10

Code Execution Successful